



AI Assisted Coding SE Led Training

Valerie Desrosier | Sr. Solution Engineer
Adam Pfister | Lead Solution Engineer



Agenda

- Intro and Level Setting with Terms, LLMs and Why We're Here
- Intro to VS Code
- Exercise: Create HTML and CSS for ArcGIS Hub with AI Assistance
- Exercise: Create a Custom ArcGIS Maps SDK for JavaScript App with an AI Assistant including the AI Components
- Optional Exercise/Demo: Create a Custom Widget with an AI Assistant for Experience Builder

Custom, vibe coded apps are coming for our accounts either way

Internal “vibe-coders”

Customer staff can spin up replacement apps with AI in an afternoon – rough, but highly visible.

Flashy competitors

Other vendors showing glossy custom solutions and imply the ArcGIS System can't keep up.

Small gaps can equal big losses

A missing widget or workflow becomes the reason a customer starts looking elsewhere but it's easier than ever for SE's to help frame how to fill that gap.

Our edge: we can make the AI produce code that is correct for the ArcGIS System — grounded in the most powerful GIS tech globally that they've already invested in, where their data lives

AI-Assisted Coding use cases for SE's

We're not the product teams or professional services, so when should we reach for AI-assisted coding in our work?

- Meeting a niche need

Customers sometimes need something specific to their industry and go to a competitor, not realizing ArcGIS can be extended to meet that need.

- Bridging a roadmap gap

When a feature is still 1-2 years out or still in discussion, custom widgets, GP tools, and apps can serve as a stop-gap that supports the customer where they are now and may even help inform product team priorities.

- Creating POCs for pre-PS engagements

Proof of concepts strengthen our PS engagements by showing customers what's possible, and we can do more of this now than before.

- Showcasing what is possible with ArcGIS and AI beyond our products

Our developer stack already does a lot, and AI extends it further. Even when something isn't available out of the box, we can push the boundaries to show what's achievable and combat flashy competitors.

Why are we here?

- So you can walk into any AI conversation and hold your own.
- Because these conversations are already happening – with or without us.
- So you can lead the AI conversation, not react to it.

Vocab Level Set

VS Code

The free editor where we'll write code and talk to Copilot. Our home base.

GitHub

Cloud home for code. Stores projects and codebases and is where you've created an account and accessed the pre-work.

Node.js

Runs JavaScript outside the browser. Powers Experience Builder and our web tooling.

localhost

Your own machine acting as a web server. "localhost:3000" = a site running only on your PC.

Markdown (.md)

A lightweight plain-text formatting syntax that uses simple symbols to style text, which then renders as formatted output.

Git

The version-control tool that tracks changes and talks to GitHub. Runs on your machine.

GitHub Copilot

The AI assistant inside VS Code. "Chat" answers questions; "Agent mode" acts on your files.

npm / npx

Node's package tools. npx runs a tool once – it's how we run tools and commands.

Languages

Python (Notebooks / ArcPy), JavaScript / TypeScript (SDK, ExB), HTML / CSS (Hub cards).

Terminal / PowerShell

A text-based interface where you type commands to control your computer directly, instead of clicking through a graphical interface.

Brief Intro to GitHub

The world's shared home for code

Think of it as a cloud home for code sharing a bit like Google Drive, but built for software and development. Its whole reason for existing is **sharing code openly**, so millions of projects are out there to read, learn from, and build on - including the skills we'll install today.

The screenshot shows the GitHub profile page for Esri. The browser address bar displays `https://github.com/Esri`. The GitHub navigation bar includes a search bar with the text "Type to search" and various utility icons. The profile header for Esri features the company logo, a "Verified" badge, and statistics: 2.6k followers, 727 repositories, 27 projects, 192 teams, and 1.3k people. Below the header, the "Pinned" section displays four repositories: `esri-leaflet` (JavaScript, 1.7k stars, 802 forks), `arcgis-rest-js` (TypeScript, 390 stars, 127 forks), `arcgis-python-api`, and `jsapi-resources`. A "View as: Public" dropdown is visible, along with a note stating "You are viewing the README and pinned repositories as a public user." A grid of user avatars is shown under the "People" section.

https://github.com/Esri

Ask Google

Esri

Type to search

Overview Repositories 727 Projects 27 Packages Teams 192 People 1.3k Security and quality Insights

esri Verified

2.6k followers Redlands, California, USA https://www.esri.com Part of Esri Inc

Follow

Pinned

`esri-leaflet` Public

A lightweight set of tools for working with ArcGIS services in Leaflet. 🚀

JavaScript 1.7k 802

`arcgis-rest-js` Public

compact, modular JavaScript wrappers for the ArcGIS REST API

TypeScript 390 127

`arcgis-python-api` Public

Documentation and samples for ArcGIS API for Python

`jsapi-resources` Public

A collection of resources for developers using the ArcGIS Maps SDK for JavaScript.

View as: Public

You are viewing the README and pinned repositories as a public user.

People

Brief Intro to GitHub

You can keep a library of them

Two directions: pulling useful code from GitHub to your local environment, and storing your own work on GitHub as you go to protect it, and keep it private or share it.

Repository ("repo")

A single project's folder of files, with its full history. Each skill set lives in its own repo – that's the `github.com/...` address you point the installer at.

Find & reuse examples

Because so much is shared openly, GitHub is where you'll discover working code to learn from – like the skills we install, plus countless ArcGIS samples.

Store your own projects

Push the things you build here. Keep a repo private while it's rough, or make it public to share with customers and colleagues.

Branches - A branch is a parallel copy of a repo where you make changes without disturbing the main version, then merge back – it's how teams work in parallel and you protect the "main" version of your code while making edits.

Forks - A fork is your own full copy of someone else's repo under your account, where you can make whatever changes you want without any access to the original. It's how you contribute to projects you don't own (make your changes on the fork, then open a pull request back to the original) and how you grab a project as a starting point for your own version.

How do LLMs work?

They predict the next words, one at a time

An AI coding assistant doesn't look things up in a database. It generates the most likely next piece of text based on patterns it saw while it was trained, which is why it makes errors

- It can be fluent and wrong

It will confidently write code that reads perfectly and doesn't run. Sounding right is not the same as being right.

- It fills gaps by guessing

Ask for an Experience Builder method it's unsure of and it may invent one that never existed.

- It has a training cutoff

It only saw the world up to a certain date. Newer ArcGIS versions, APIs, and deprecations can be invisible to it.

- It starts each chat blank

By default it remembers nothing from your last session. It only works with what's in front of it now.

How do LLMs work?

It can only work with what's in the current chat - it's "Context Window"

Think of it like being in a long meeting. You can keep the last stretch of conversation in your head, but you can't recall every sentence from three hours ago. The early details get crowded out by everything said since.

What it can see

Everything in the current chat: your prompt, the files you've shared, and its own earlier replies. Nothing else.

There's a fixed limit

That working space has a hard size. Once you fill it, the oldest content gets pushed out to make room.

Long chats drift

The more a single chat runs on, the more the early instructions get buried, answers start to wander, the chat becomes slower and more expensive

In VS Code - a new task should always equal a new chat

(Tip: if you're unsure about switching context windows, just mention you'll be opening a new chat and ask for the assistant to help you do a "handoff")

Updated Aug 2026

The frontier models, August 2026

Hosted frontier model providers

OpenAI | GPT-5.6

Sol • Terra • Luna

Best at: Sol for agentic coding and deep reasoning; Terra/Luna for balanced and low-cost workflows.

Anthropic | Claude 5

Fable 5 • Mythos 5 • Opus 5 • Sonnet 5

Best at: careful coding, long-context reviews, spec-following, enterprise-grade agent workflows.

Google DeepMind | Gemini 3.x

Gemini 3.7 Flash • 3.1 Pro Preview • 3.1 Live / TTS / Omni

Best at: multimodal reasoning, fast/cost-scaled apps, Google ecosystem and geospatial-adjacent workflows.

SpaceXAI | Grok 4.6

grok-4.6

Best at: coding, agentic tool use, knowledge work, and large-context IDE or workflow agents.

Open-source / open-weight leaders

Meta | Muse

Spark • Glimmer

Best at: coding-optimized frontier model, especially strong at agentic coding, multi-file software work, computer use, and multimodal workflows.

Mistral AI | 3 / 3.5 / 4

Mistral Large 3 • Medium 3.5 • Small 4

Best at: open-weight enterprise deployment, agentic coding, multimodal tasks, and European data-control options.

DeepSeek | V4

DeepSeek-V4-Pro • V4-Flash

Best at: cost-effective reasoning and coding, long context, and efficient mixture-of-experts inference.

Alibaba Qwen | Qwen 3.8

Qwen3.8-Max

Best at: coding, multilingual work, multimodal agents, and a wide open model ecosystem.

NVIDIA | Nemotron 3

Ultra • Super • Nano

Best at: open agentic models, high-throughput inference, and specialized enterprise AI workflows.

Common Problems: Why can't I get it to work?

1 *It didn't build what I wanted*

You are thinking one thing, it builds another. The same communication gap between humans with little planning.

Fix: grill the design first. Make the AI interview you with detailed questions before it writes a line.

2 *It's far too verbose*

Dropped into a project with no context, it doesn't know your terms, so it uses twenty words where one would do.

Fix: a shared vocabulary. A short project glossary the AI reads, so you both speak the same language.

3 *The code doesn't work*

Even when you're clear, it can still produce junk – because it never sees the results of its code.

Fix: use skills to run real feedback loops. Working through AI-assisted coding will be an iterative process.

4 *We built a ball of mud*

AI speeds up coding, which also speeds up mess. Codebases get tangled and hard to change, fast.

Fix: care about design continuously. Check which pieces you're touching, and review before shipping.

The solution: markdown files and skills

Markdown files are the plain text the AI reads like a person

A .md file is just text with a few light formatting marks. No special app – it opens in any editor. It's the standard way to write instructions for AI coding tools.

- # Heading *structure*
- ****bold**** for emphasis *signals importance*
- - bullet lists *steps & options*
- `code` blocks *exact commands*
- [links](url) *point to real docs*

Why Markdown, not Word?

Lightweight – takes up very little of the AI's working space.

Structured – headings and lists tell it how to read the file.

Versionable – lives in Git right next to your code.

Portable – the same file works across Copilot, Claude, and other tools.

The solution: markdown files and skills

A skill is a folder that teaches the AI a job

At its core it's a SKILL.md file: a name, a description of when to use it, and step-by-step instructions. It can bring scripts and examples along too.

SKILL.md

```
name: exb-widget
description: when to reach for it
_____

# The instructions
step-by-step procedure the
AI follows every time
```

Consistency

The same correct procedure runs every time – not whatever the AI happened to do today.

Expertise on tap

Encodes hard-won knowledge: version rules, safety checks, the right documentation source.

Reusable & shareable

Write once, drop into any project, hand to a teammate. Works across AI tools.

Skills load in stages – only what's needed

You can keep a library of them

Because the AI's working space is limited, skills don't all load at once. The AI pulls each one in three stages, and only reaches the next stage when the task actually calls for it.

1

Always available

Just the name and a one-line description of each skill. Tiny. The AI scans these to judge relevance.

2

Loaded when it matches

Your request matches a description → the full SKILL.md instructions come into the chat.

3

Pulled only if used

A script or example file inside the skill loads only if the instructions actually reach for it.

This is what makes a whole library of ArcGIS skills practical – install as many as you like, and the AI only pulls in the ones a task needs.

Skills vs. custom instructions

■ Agent Skills

A capability, loaded on demand

- Teaches a specific job or workflow
- Can carry scripts, examples, resources
- Loads only when your task matches
- Portable across Copilot, Claude, others
- Lives in `.github/skills` or `.agents/skills`

■ Custom Instructions

Standing house rules, always on

- Project-wide coding standards & conventions
- Instructions only – no scripts
- Applied to every request (or by file pattern)
- e.g. “never cite ArcMap; use `pro.arcgis.com`”
- Lives in `.github/copilot-instructions.md`

Vibe Coding → AI Assisted Coding



Matt Pocock's skills: the engineering discipline

github.com/mattpocock/skills

Small, simple skills for real engineering and development to ensure your AI assistant isn't "vibe coding"

ask-matt

A router — tells you which skill or flow fits the situation you're in.

grill-with-docs

Interviews you to nail the design, and builds your project's shared vocabulary as it goes.

domain-modeling / CONTEXT.md

Builds and sharpens a shared project language so you and the AI stop talking past each other.

wayfinder

Maps a huge chunk of work as decision tickets, resolved one at a time until the path is clear.

diagnosing-bugs

A disciplined loop for hard bugs: reproduce → minimize → hypothesize → fix → regression-test.

to-spec

Turns the conversation you just had into a written spec, published to your issue tracker.

to-tickets

Breaks a plan or spec into small tickets, each declaring what it's blocked by.

improve-codebase-architecture

Surveys the codebase for cleanup opportunities and grills through the one you pick.

Matt Pocock's skills: the engineering discipline

github.com/mattpocock/skills

1 grill-with-docs
OR
wayfinder

2 to-spec

3 to-tickets

4 implement

5 code-review



ArcGIS skills: adding Esri expertise

github.com/valdesrosier/arcgis-skills

Built to work alongside Matt's set or by itself. Each one loads on its own: describe your ArcGIS task in plain language and the right skill applies automatically.

arcgis-docs-lookup

Routes to the authoritative Esri source, version-scoped – never ArcMap or Desktop.

python-notebook

ArcGIS API for Python for hosted Notebooks or local, with a check before destructive edits.

arcade

Correct Arcade profile (popup, labeling, rules); verifies every function exists at your version.

exb-widget

Experience Builder custom widgets at the correct compile version; checks the Node / pnpm gate.

js-sdk

Maps SDK for JavaScript – knows the 3.x / 4.x / 5.x boundaries, avoids the dead AMD path.

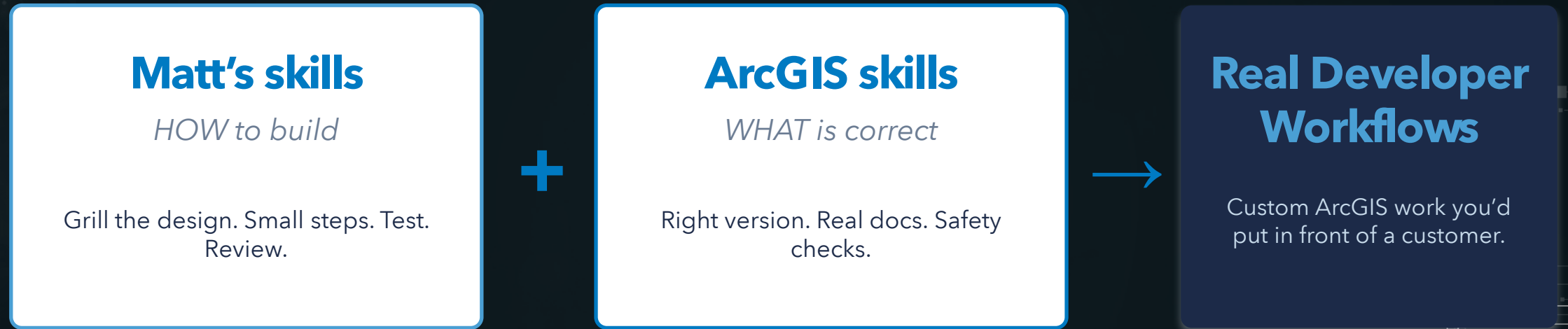
arcgis-custom-data-feeds

Koop / Node providers exposing external systems as Feature Services, version-pinned.

arcgis-html-css

HTML CSS follows the guidelines required to develop for ArcGIS Hub and Popups.

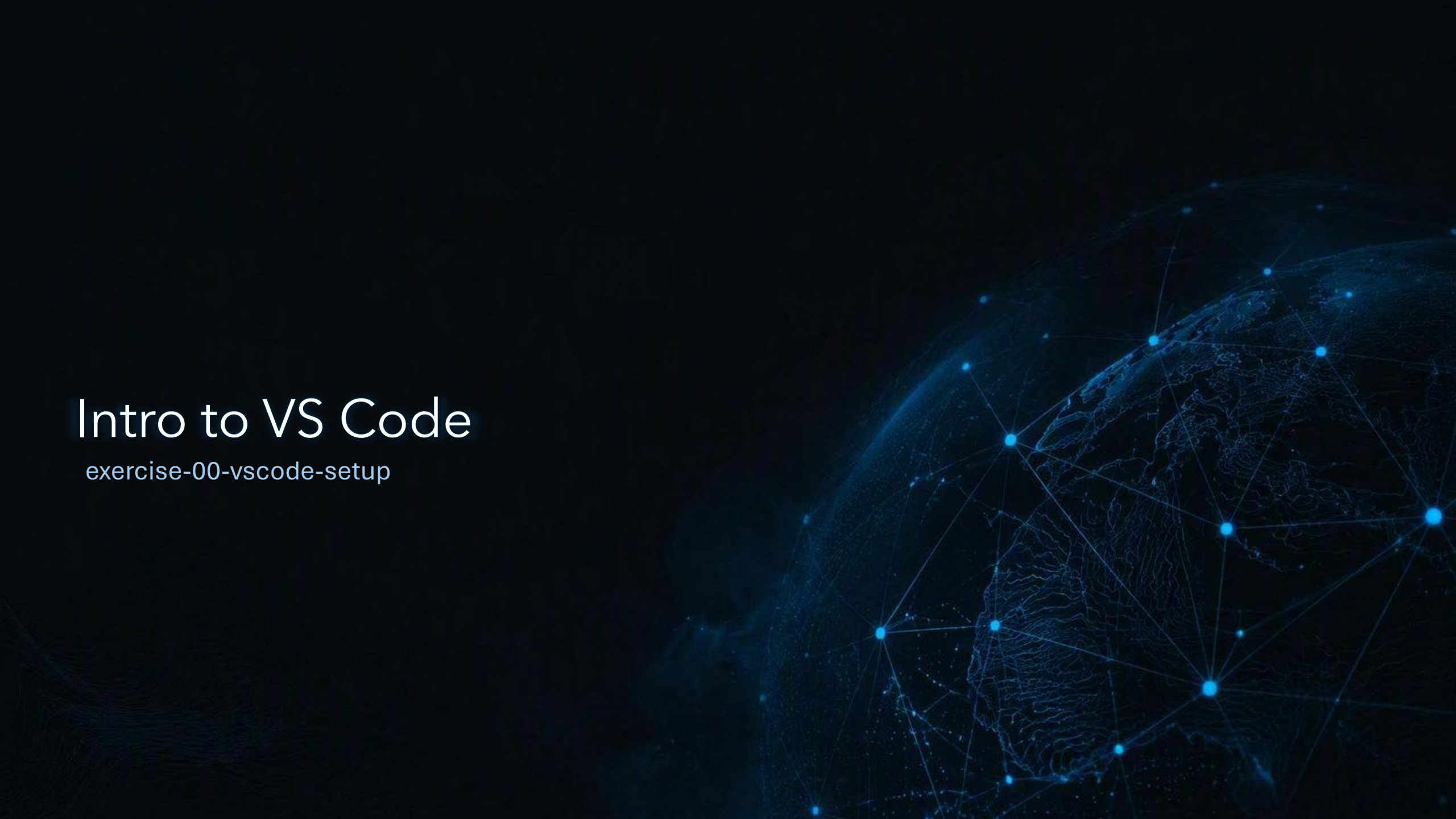
Discipline and domain takes “vibe code” to AI-assisted coding



Today we'll do this three times: HTML and CSS for Hub, an ArcGIS Maps for JavaScript SDK App, and an Experience Builder widget.

Intro to VS Code

exercise-00-vscode-setup



10 min break



HTML/CSS for Hub Workflow

exercise-01-htmlcss



ArcGIS Maps for JavaScript SDK App Build

exercise-02-javascriptsdk



Experience Builder Widget Workflow

exercise-04-experiencebuilder



When something breaks, or you want to add more

What's Next – keep going after today

You've built the core. Two habits keep the work healthy once you're on your own.

When something breaks

Don't just say "it's not working." Ask the agent to use `/diagnosing-bugs`.

It runs a disciplined loop:

reproduce → **shrink to the smallest failing case** → **form a hypothesis** → **test** → **fix** → **add a check so it can't come back**

Overkill for a typo. Perfect for a "why is this behaving strangely" moment where a plain "fix it" sends the AI in circles.

When you want to add a feature

Don't bolt it on mid-conversation. Start a new `/grill-with-docs session` scoped to just the new piece – the same design-first flow you used the first time.

Why it works:

- The addition gets the same careful interview the original did.
- You start on a clean slate – not a long, cluttered chat where earlier decisions are half-forgotten.

Start a fresh chats – and carry your work over

What's Next – keep going after today

When to start a new chat

A chat has a limited working space. A long one fills up – early instructions get buried and answers start to wander.

The signs:

- it forgets a decision you made earlier
- its replies get vaguer
- it starts going in circles

Start fresh before degradation sets in – don't push a tired chat further.

Carry it over with `/handoff`

It compacts the conversation into a short handoff doc – what you'd want a new teammate to read before picking up your work. A good one captures:

- **What you're building** – the goal, in a sentence or two
- **Decisions already made** – so they don't get re-litigated
- **Where you left off** – what's built and working so far
- **What's next** – the immediate next step

Open a fresh chat, point it at the handoff doc, and you're back to full context on a clean slate.

Every message costs – spend it wisely

Use AI Responsibly

These tools run on tokens – the text the model reads and writes. More text means more cost and slower replies. A few habits keep your usage lean.

■ Default to the auto model

In GitHub Copilot, let it pick the model (Auto) for everyday work. Reach for a bigger, pricier model only when a task actually needs it – not by default.

■ Watch your context window

The bigger a chat gets, the more tokens every new reply costs, because the whole conversation is re-read each time. Long chats are expensive chats.

■ Start fresh, stay scoped

Open a new chat when one balloons, and keep each session focused on one task. A tidy context is cheaper and gives better answers.

■ Let skills do the heavy lifting

Skills load only what a task needs instead of you pasting huge docs into the chat – that's less token burn and more consistent results.

Keep your skills up to date

skills aren't set and forget

Skills evolve – Matt refines his set constantly, and the ArcGIS skills track new versions and API changes. A skill from three months ago may be out of date.

1

Re-run the installer to update

The same command that installs a skill pulls the latest version. Re-run it now and then to stay current.

2

Watch for ArcGIS version bumps

When your org moves to a new Enterprise or Maps SDK version, refresh the ArcGIS skills so their guidance matches.

3

Skills are yours to edit

They're just markdown. If a skill doesn't quite fit your workflow, open it and adjust it – or write your own.

To update: re-run `npx skills@latest add mattpocock/skills` (and the same for the ArcGIS skills) to pull the newest versions.

Where to go next

Explore more on your own

Where to learn more:

- Watch Matt's tutorial videos: aihero.dev
- Tutorial to run through ExB Widget development again:
<https://www.youtube.com/watch?v=NuxNCvL0UUA>

Other skills to explore:

- impeccable.style - design skills to iterate on your apps without looking like "AI slop" | <https://impeccable.style/>
- bro-skill - re-explains the last answer in plain language to help you learn | <https://github.com/luchasarie/bro-skill>

Design your own skills with your copilot assistant:

- Use `/writing-for-agents` - Matt's skill to help you write skills
- Design a great skill?? Share!

Five ideas to remember about AI-Assisted Coding

- 1 Use AI as an engineering partner, not an auto-complete engine.
- 2 Give the AI context, not just commands.
- 3 Design before you build. The grilling/planning session is where the real work happens.
- 4 Skills are how you make it an expert.
- 5 **Matt's engineering skills + ArcGIS expertise skills = developer level workflows**